

PORTADA

Arquitectura de Software

Semana 16 — Principios de Arquitectura de Software Segura

Cuando la seguridad no es una capa que se agrega al final — apertura de la Unidad 5 — con el caso real de Equifax (2017) · Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



147 millones de personas. Un parche que existía desde marzo y nunca se aplicó.

Equifax, 2017 — la brecha de datos más citada de la industria como ejemplo de que la seguridad es una decisión arquitectónica, no un parche de último momento.



TRANSICIÓN

De una de las tres mayores agencias de crédito de Estados Unidos a la brecha más investigada de la década

Equifax es una de las tres principales agencias de reporte crediticio de Estados Unidos, custodia de datos financieros altamente sensibles — números de seguro social, fechas de nacimiento, direcciones y números de tarjeta de crédito — de cientos de millones de personas.

En septiembre de 2017, Equifax reveló que había sido víctima de una de las brechas de datos más grandes de la historia — y la investigación posterior del Congreso de EE. UU. y de la Oficina de Rendición de Cuentas del Gobierno (GAO) documentó, paso a paso, cómo fallaron los controles arquitectónicos de seguridad, no solo un error humano aislado.

La brecha de datos de Equifax

Empresa	Equifax Inc.
Personas afectadas	Aproximadamente 147 millones, principalmente en EE. UU., además de residentes de Reino Unido y Canadá
Datos expuestos	Nombres, números de seguro social, fechas de nacimiento, direcciones y, en algunos casos, números de licencia de conducir y de tarjeta de crédito
Vulnerabilidad explotada	Falla conocida en el framework Apache Struts (CVE-2017-5638)
Acceso inicial	13 de mayo de 2017, a través del portal de disputas en línea
Descubrimiento / divulgación	Descubierta internamente el 29 de julio de 2017, divulgada públicamente el 7 de septiembre de 2017
Investigación	Comité de Supervisión de la Cámara de Representantes de EE. UU. y Oficina de Rendición de Cuentas del Gobierno (GAO), 2018

CASO · LA CADENA DE FALLAS

Un parche disponible desde marzo, aplicado nunca

El fabricante del framework Apache Struts publicó un parche para la vulnerabilidad CVE-2017-5638 el **7 de marzo de 2017**. Equifax tenía un proceso interno para identificar y aplicar parches críticos — y ese proceso falló en detectar que el portal de disputas en línea usaba la versión vulnerable.

Dos meses después, el 13 de mayo de 2017, atacantes explotaron exactamente esa vulnerabilidad sin parchear para obtener acceso inicial al sistema.

CASO · POR QUÉ NADIE LO NOTÓ DURANTE 76 DÍAS

Un certificado de seguridad vencido dejó ciego al monitoreo

Una vez dentro, los atacantes se movieron lateralmente hacia bases de datos con información sensible — algo que la investigación de la GAO atribuyó a una **red plana, sin segmentación adecuada** entre el sistema expuesto a internet y los sistemas internos con datos críticos.

Equifax sí tenía un dispositivo para inspeccionar tráfico cifrado en busca de actividad sospechosa — pero el certificado digital de ese dispositivo llevaba **cerca de 19 meses vencido**, por lo que no podía descifrar ni inspeccionar el tráfico. Los atacantes exfiltraron datos sin ser detectados durante **76 días**, hasta que el certificado se renovó el 29 de julio de 2017 y el tráfico sospechoso finalmente se hizo visible.

TRANSICIÓN

Cuatro fallas, cuatro principios de arquitectura segura que las habrían evitado

La investigación oficial resumió la brecha en cuatro causas raíz: **identificación** (no se identificó la necesidad de parchear), **detección** (el certificado vencido cegó el monitoreo), **segmentación** (una red plana permitió el movimiento lateral) y **gobernanza de datos** (se retuvieron más datos sensibles de los necesarios). Cada una de esas fallas tiene un principio arquitectónico formal que la aborda — los que vemos a continuación.

CONCEPTO

La tríada CIA: el modelo base de la seguridad de la información

Todo principio de seguridad que veremos hoy protege, en el fondo, una de estas tres propiedades — el modelo clásico documentado por el NIST (Instituto Nacional de Estándares y Tecnología de EE. UU.):

- **Confidencialidad:** solo quienes están autorizados pueden acceder a la información (en Equifax, los datos de 147 millones de personas quedaron expuestos a atacantes no autorizados).
- **Integridad:** la información no puede ser alterada sin autorización ni detección.
- **Disponibilidad:** los sistemas y datos están accesibles cuando quienes están autorizados los necesitan.

La brecha de Equifax fue, sobre todo, una falla masiva de confidencialidad — pero las tres propiedades son responsabilidad de la arquitectura, no solo de un equipo de seguridad separado.

CONCEPTO

Security by Design: la seguridad se decide en la arquitectura, no se agrega después

En 1975, Jerome Saltzer y Michael Schroeder publicaron "**The Protection of Information in Computer Systems**", un artículo que formalizó ocho principios de diseño para construir sistemas seguros desde su arquitectura — sigue siendo, medio siglo después, la referencia académica más citada del campo.

Principio	Qué exige
Fail-safe defaults	Si algo falla, el sistema debe negar el acceso por defecto, no concederlo.
Complete mediation	Cada solicitud de acceso debe verificarse, sin excepciones ni atajos.
Economy of mechanism	Mientras más simple el diseño de seguridad, menos superficie de error.

CONCEPTO

Mínimo privilegio: cada componente opera con lo justo, nunca con más

El **principio de mínimo privilegio** (least privilege), también formalizado por Saltzer y Schroeder (s09), exige que cada usuario, proceso o servicio tenga acceso únicamente a los recursos estrictamente necesarios para cumplir su función — nada más.

Ejemplo aplicado: la cuenta de servicio que usa la aplicación web para consultar el catálogo de productos no debería tener permisos para modificar la tabla de usuarios ni para leer datos de facturación — aunque técnicamente comparta la misma base de datos.

Un atacante que compromete un componente con privilegios mínimos solo hereda esos privilegios mínimos — el daño queda contenido.

CONCEPTO

Defensa en profundidad: varias capas, ninguna es la única línea de defensa

La **defensa en profundidad** es un principio arquitectónico, de origen militar, adaptado a la seguridad de la información: en vez de confiar en un solo control (ej. un firewall perimetral), se despliegan varias capas independientes, de modo que si una falla, las siguientes aún protegen el sistema.

- Capas típicas: perímetro de red, segmentación interna, control de acceso a nivel de aplicación, cifrado de datos, monitoreo y respuesta a incidentes.

Cada capa debe ser independiente: si comprometer una automáticamente compromete a las demás, no es defensa en profundidad real.

CONTRAEJEMPLO

Equifax frente a la defensa en profundidad

Capa	Qué debía existir	Qué falló en Equifax
Parcheo	Aplicar el parche de Struts disponible desde marzo de 2017	El proceso de identificación de vulnerabilidades no marcó el sistema como afectado
Segmentación	Aislar los sistemas con datos sensibles del portal expuesto a internet	Una red plana permitió el movimiento lateral sin restricciones
Monitoreo	Inspeccionar tráfico cifrado en busca de exfiltración	Certificado vencido durante ~19 meses dejó el tráfico sin inspeccionar
Gobernanza de datos	Retener y proteger solo los datos estrictamente necesarios	Se conservaron más datos sensibles de los necesarios

Cuatro capas de defensa en profundidad — las cuatro fallaron de forma independiente, justo lo que este principio busca evitar.

CONCEPTO

Zero Trust: nunca confiar, siempre verificar

En 2010, el analista **John Kindervag**, de Forrester Research, acuñó el término **Zero Trust** para describir un modelo que abandona la idea de un "perímetro confiable": ninguna solicitud, venga de dentro o fuera de la red corporativa, se confía por defecto. En 2020, el NIST formalizó el modelo en el estándar **SP 800-207** (Zero Trust Architecture).

- Verificar explícitamente cada solicitud.
- Aplicar mínimo privilegio (s10) de forma consistente.
- Asumir que la brecha ya ocurrió ("assume breach") y diseñar el sistema para limitar el daño.

La red plana de Equifax (s06) es el opuesto exacto de Zero Trust: una vez dentro del perímetro, un atacante se movía sin restricciones adicionales.

TENSIÓN

Seguridad perimetral tradicional vs. Zero Trust

	Seguridad perimetral	Zero Trust
Supuesto base	"Dentro de la red = confiable"	Ninguna solicitud es confiable por defecto
Punto de verificación	Solo en el borde de la red (firewall)	En cada solicitud, sin importar el origen
Movimiento lateral de un atacante	Prácticamente libre una vez dentro	Restringido — cada salto requiere verificación
Adecuado para	Redes cerradas, sin nube ni trabajo remoto	Arquitecturas modernas: nube, microservicios, APIs

La mayoría de las arquitecturas modernas (los estilos vistos en la Unidad 2) ya no tienen un perímetro claro que defender — de ahí la adopción creciente de Zero Trust.

CONCEPTO

Threat modeling: anticipar las amenazas antes de escribir código

El **threat modeling** es el proceso de identificar, de forma sistemática, las amenazas posibles a un sistema durante su diseño — antes de que exista una sola línea de código en producción. La metodología más usada en la industria es **STRIDE**, desarrollada por Loren Kohnfelder y Praerit Garg en Microsoft en 1999.

Categoría STRIDE	Qué representa
Spoofing (suplantación)	Alguien se hace pasar por otro usuario o sistema
Tampering (alteración)	Modificar datos o código sin autorización
Repudiation (repudio)	Negar haber realizado una acción, sin poder probarlo
Information disclosure	Exponer información a quien no debería verla
Denial of service	Impedir que el sistema esté disponible para usuarios legítimos
Elevation of privilege	Obtener permisos mayores a los concedidos

INTERACCIÓN

Apliquen STRIDE a su proyecto de CodeF@ctory

En equipos:

1. Elijan un componente crítico de su sistema (ej. login, pagos, carga de archivos).
2. Para cada categoría de STRIDE (s15), identifiquen si existe una amenaza plausible concreta contra ese componente.
3. Para la amenaza más crítica que encuentren, propongan un control (mínimo privilegio, defensa en profundidad, Zero Trust) que la mitigue.

10 minutos · respondan en equipo, prepárense para compartir un hallazgo

CONCEPTO EN DISPUTA

"Ya casi terminamos, la seguridad la agregamos después"

Es una frase común en proyectos con presión de tiempo — y es exactamente lo que este principio contradice. Agregar seguridad al final casi siempre significa parchear síntomas (validar una entrada aquí, agregar un chequeo allá) sobre una arquitectura que nunca fue diseñada para resistir ataques, en vez de tener controles integrados desde las decisiones estructurales.

El caso de Equifax no fue causado por la ausencia de un equipo de seguridad — Equifax tenía uno. Fue causado por decisiones y procesos arquitectónicos (parcheo, segmentación, monitoreo, gobernanza de datos) que no sostuvieron esas capas cuando fallaron.

Esta es una tensión real y documentada en la industria, no una simplificación — la retrospectiva es la prueba constante de qué tan cara sale "agregarla después".

SÍNTESIS

Ideas clave de hoy

1. La brecha de Equifax (2017, 147 millones de personas) es el caso más citado de que la seguridad es una decisión arquitectónica: fallaron el parcheo, la segmentación, el monitoreo y la gobernanza de datos, no un solo control aislado.
2. La tríada CIA (confidencialidad, integridad, disponibilidad) es el modelo base que todo principio de seguridad protege.
3. Los principios de Saltzer y Schroeder (1975) — fail-safe defaults, mínimo privilegio, mediación completa, entre otros — siguen siendo la base académica de Security by Design.
4. La defensa en profundidad exige varias capas independientes; Zero Trust (Kindervag, 2010; NIST SP 800-207) reemplaza la confianza implícita del perímetro por verificación explícita en cada solicitud.
5. El threat modeling con STRIDE (Microsoft, 1999) permite anticipar amenazas desde el diseño, antes de escribir código.

CIERRE

Para la próxima sesión

- Repasar los principios de hoy (CIA, Security by Design, mínimo privilegio, defensa en profundidad, Zero Trust, STRIDE) y ubicar cuáles aplica o podría aplicar su proyecto de CodeF@ctory.
- Tener presente que la seguridad de este semestre se evalúa en el hito "Seguridad Arquitectónica y Hardening" (20%), programado en esta misma semana.

La próxima sesión cerramos la Unidad 5 bajando estos principios a un terreno muy concreto: la seguridad en las APIs — la puerta de entrada que la mayoría de los sistemas modernos exponen al mundo.

